O Desafio do Oráculo Preditivo

Projeto

Este projeto foi elaborado para a Matéria **Aprendizagem de Máquina**.

Professor: Medina

Alunos:

- Gustavo Bravo
- Gustavo Martins
- Lucas
- Thiago

Natureza do Problema: Aprendizado Supervisionado - Regressão

O problema abordado é de **Aprendizado Supervisionado**.

No Aprendizado Supervisionado (Supervised Learning), o algoritmo aprende a partir de exemplos rotulados, ou seja, dados que contêm tanto as variáveis de entrada (atributos ou X) quanto a resposta correta (rótulo/alvo ou Y). O objetivo do algoritmo é descobrir a regra matemática que mapeia X em Y para fazer previsões em novos dados.

Detalhes do Conjunto de Dados `Health_LifeStyle-DS.csv`:

- **Entradas (X)**: `Age`, `BMI`, `Exercise_Frequency`, `Diet_Quality`, `Sleep_Hours`, `Smoking_Status` e `Alcohol_Consumption`.
- **Saída Conhecida (Y)**: `Health_Score`.

Como a base já possui a "resposta correta" (`Health_Score`) para cada indivíduo do histórico, o modelo é treinado de forma supervisionada.

Tipos de Problemas no Aprendizado Supervisionado:

Dentro do Aprendizado Supervisionado, existem dois tipos principais de problemas:

- **Classificação**: A variável alvo (Y) é categórica ou discreta — o modelo tenta prever uma classe ou categoria (exemplo: "Spam" ou "Não Spam", "Sobreviveu

1" ou "Não Sobreviveu 0").
- **Regressão**: A variável alvo (Y) é numérica contínua ou real — o modelo tenta prever uma quantidade ou valor contínuo (exemplo: preço de um imóvel, salário, temperatura ou faturamento).

O dataset `Health_Score` assumirá valores numéricos contínuos em uma escala real (ex: 70.54, 57.24, 96.33, etc.). Como o objetivo é prever um valor numérico contínuo e não categorizar o paciente em um grupo fixo, o problema é estritamente de **Regressão**.

1) Resumo do Projeto

1.1. O Problema

O objetivo do projeto é prever a pontuação de saúde de indivíduos com base em seus hábitos de vida.

1.2. A Metodologia

- **Dados Utilizados**: `Health_LifeStyle-DS.csv`
- **Algoritmos Treinados**: Regressão Linear Múltipla, Random Forest Regressor, SVR e Dummy Regressor.

1.3. Os Resultados Principais

Será determinado qual modelo obteve o maior R^2 e os menores erros (MAE/RMSE).

1.4. A Conclusão

Será recomendada a melhor abordagem para uso prático com base nos resultados.

2) Introdução e Contextualização

Contextualização do Problema:

Na área da saúde preventiva e bem-estar, a avaliação do estado geral de saúde de um indivíduo frequentemente depende de uma combinação complexa entre dados biométricos e hábitos de vida diários. Este projeto utiliza o conjunto de dados **Health and Lifestyle Data** para analisar como fatores comportamentais (como sono, dieta, exercícios, tabagismo e consumo de álcool) e biométricos (como idade e IMC) impactam quantitativamente o estado de saúde de uma pessoa.

A construção de um modelo preditivo capaz de estimar esse índice permite antecipar riscos à saúde e orientar intervenções preventivas personalizadas antes do surgimento de complicações graves.

Mapeamento da Variável Alvo (Target — Y):

Health_Score: Pontuação numérica contínua que quantifica o indicador geral de saúde do indivíduo. Como o objetivo é prever uma quantidade real contínua a partir de dados históricos rotulados, o problema enquadra-se estritamente como Aprendizado Supervisionado de Regressão.

Mapeamento das Variáveis Preditoras (Features — X):

Para prever o Health_Score, o modelo utiliza 7 atributos preditores extraídos da base de dados:

- **Age**: Idade do indivíduo em anos (variável numérica contínua).
- **BMI**: Índice de Massa Corporal em $\mathrm{kg/m^2}$ (variável numérica contínua).
- **Exercise_Frequency**: Frequência semanal de prática de exercícios físicos em dias (variável numérica discreta).
- **Diet_Quality**: Métrica de avaliação da qualidade da alimentação (variável numérica contínua).
- **Sleep_Hours**: Média de horas de sono por noite (variável numérica contínua).
- **Smoking_Status**: Indicador de tabagismo (variável binária: 0 para não fumante, 1 para fumante).
- **Alcohol_Consumption**: Nível semanal de consumo de bebidas alcoólicas (variável numérica contínua).

3) Objetivos do Projeto

Objetivo Geral:

Realizar uma análise comparativa e rigorosa entre três abordagens de Aprendizado Supervisionado de Regressão — Regressão Linear Múltipla, Random Forest Regressor (RFR) e Support Vector Regressor (SVR) — para prever o indicador contínuo de saúde (Health_Score) a partir de hábitos de vida e atributos biométricos. Busca-se identificar a técnica com maior poder de generalização e explicar as razões de sua performance frente à natureza dos dados.

Objetivos Específicos:

1. **Análise Exploratória e Tratamento de Dados (EDA)**: Mapear a distribuição da variável alvo (`Health_Score`), identificar correlações com os hábitos de vida e avaliar a presença de outliers e risco de multicolinearidade.

2. **Pré-processamento e Escalonamento**: Aplicar padronização/normalização adequada às features preditoras (requisito crítico para modelos baseados em distância/kernel como o SVR) e realizar a divisão dos dados entre conjuntos de treino e teste com validação cruzada.
3. **Treinamento e Ajuste de Modelos**: Implementar e ajustar a Regressão Linear Múltipla (como baseline estatístico), o Random Forest Regressor (para capturar relações não-lineares) e o SVR (testando a robustez com kernels).
4. **Validação Comparativa**: Avaliar o desempenho dos três algoritmos por meio de uma tabela comparativa consolidando as métricas R^2, MAE, MAPE (%) e RMSE, além da análise do gráfico de resíduos e do painel "Valores Reais vs. Preditos".
5. **Implantação Interativa**: Desenvolver um dashboard executivo no Streamlit que permita simular o Health_Score de um indivíduo em tempo real de acordo com as alterações nos seus hábitos diários.

4) Análise Exploratória de Dados (EDA)

4.1 Carregamento, Inspeção Inicial e Diagnóstico da Base

Nesta primeira etapa, carregamos a base de dados `Health_LifeStyle-DS.csv` e realizamos uma inspeção estatística e estrutural para verificar as dimensões do dataset, os tipos de dados, a presença de valores ausentes e registros duplicados.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
# bibliotecas utilizadas
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# estilo visual
sns.set_theme(style="whitegrid")
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

import time
```

```
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error,
from sklearn.model_selection import cross_val_score
```

```
##health_df = pd.read_csv("~/Área de trabalho/FATEC_nuvem/lab_ML_2026,
health_df = pd.read_csv("/content/sample_data/Health_LifeStyle-DS.csv
# dimensão (linhas, colunas)
health_df.shape
```

```
(1000, 8)
```

```
# primeras linhas
health_df.head()
```

	Age	BMI	Exercise_Frequency	Diet_Quality	Sleep_Hours	Sm
0	45.960570	31.996777	5	55.403270	7.300359	
1	38.340828	29.623168	6	41.838357	7.012419	
2	47.772262	25.298152	5	76.904948	6.028641	
3	58.276358	21.765316	2	49.756767	5.802714	
4	37.190160	28.491117	2	44.218737	7.912548	

Próximas etapas: Gerar código com health_df | New interactive sheet

```
# tipos de dados e nulos
health_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 8 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Age                  1000 non-null   float64
 1   BMI                  1000 non-null   float64
 2   Exercise_Frequency   1000 non-null   int64
 3   Diet_Quality         1000 non-null   float64
 4   Sleep_Hours          1000 non-null   float64
 5   Smoking_Status       1000 non-null   int64
 6   Alcohol_Consumption  1000 non-null   float64
 7   Health_Score         1000 non-null   float64
dtypes: float64(6), int64(2)
memory usage: 62.6 KB
```

```
# estatistica descritiva
health_df.describe().T
```

	count	mean	std	min	25%	
Age	1000.0	40.231985	11.750591	1.104792	32.228916	40.303
BMI	1000.0	25.354181	4.987272	10.298057	21.968792	25.315
Exercise_Frequency	1000.0	2.888000	1.995354	0.000000	1.000000	3.000
Diet_Quality	1000.0	69.952977	14.972061	19.907497	59.945481	69.975
Sleep_Hours	1000.0	6.973135	1.517218	2.431107	5.903351	6.990
Smoking_Status	1000.0	0.499000	0.500249	0.000000	0.000000	0.000
Alcohol_Consumption	1000.0	3.079377	2.084564	-3.592506	1.644111	3.064
Health_Score	1000.0	85.479947	13.633845	29.106017	76.430819	87.498

```
# valores ausentes
health_df.isnull().sum()
```

	0
Age	0
BMI	0
Exercise_Frequency	0
Diet_Quality	0
Sleep_Hours	0
Smoking_Status	0
Alcohol_Consumption	0
Health_Score	0

dtype: int64

```
# valores duplicados
health_df.duplicated().sum()
```

```
np.int64(0)
```

Estrutura: O dataset possui **1.000 linhas** e **8 colunas** numéricas contínuas/discretas. **Integridade:** Não há valores nulos nem registros duplicados. A base está 100% preenchida e pronta para a análise.

4.2 Análise Univariada da Variável Alvo (`Health_Score`)

Analisamos o comportamento da variável dependente `Health_Score` para identificar sua tendência central, dispersão e se a sua distribuição de frequência se

aproxima de uma distribuição normal ou apresenta assimetria.

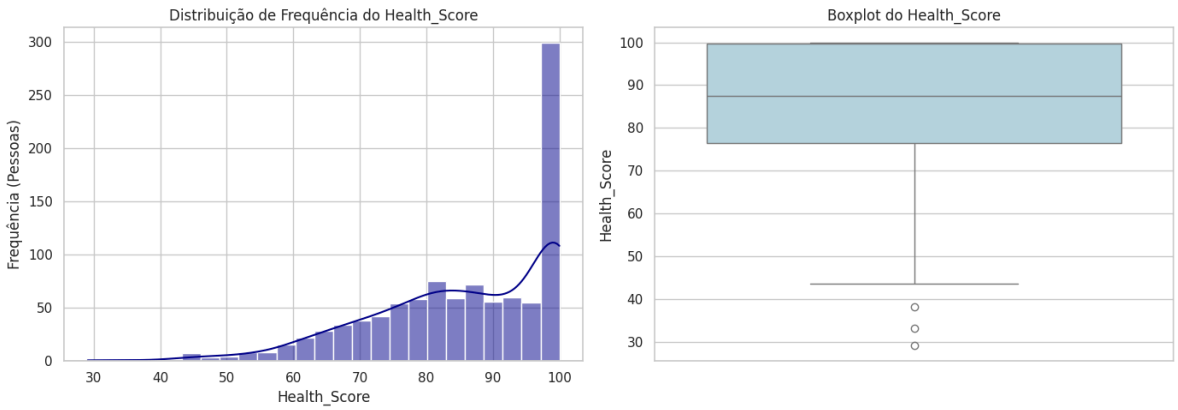
```
# Criando um painel visual com Histograma e Boxplot para o Health_Sco
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Histograma com curva de densidade (KDE)
sns.histplot(health_df['Health_Score'], kde=True, color='darkblue', b
axes[0].set_title('Distribuição de Frequência do Health_Score', fonts
axes[0].set_xlabel('Health_Score')
axes[0].set_ylabel('Frequência (Pessoas)')

# Boxplot da Variável Alvo
sns.boxplot(y=health_df['Health_Score'], color='lightblue', ax=axes[1
axes[1].set_title('Boxplot do Health_Score', fontsize=12)
axes[1].set_ylabel('Health_Score')

plt.tight_layout()
plt.show()

# Hipótese de Negócio / Estatística
print(f"Média do Health_Score: {health_df['Health_Score'].mean():.2f}
print(f"Mediana do Health_Score: {health_df['Health_Score'].median():
```




```
Média do Health_Score: 85.48
Mediana do Health_Score: 87.50
```

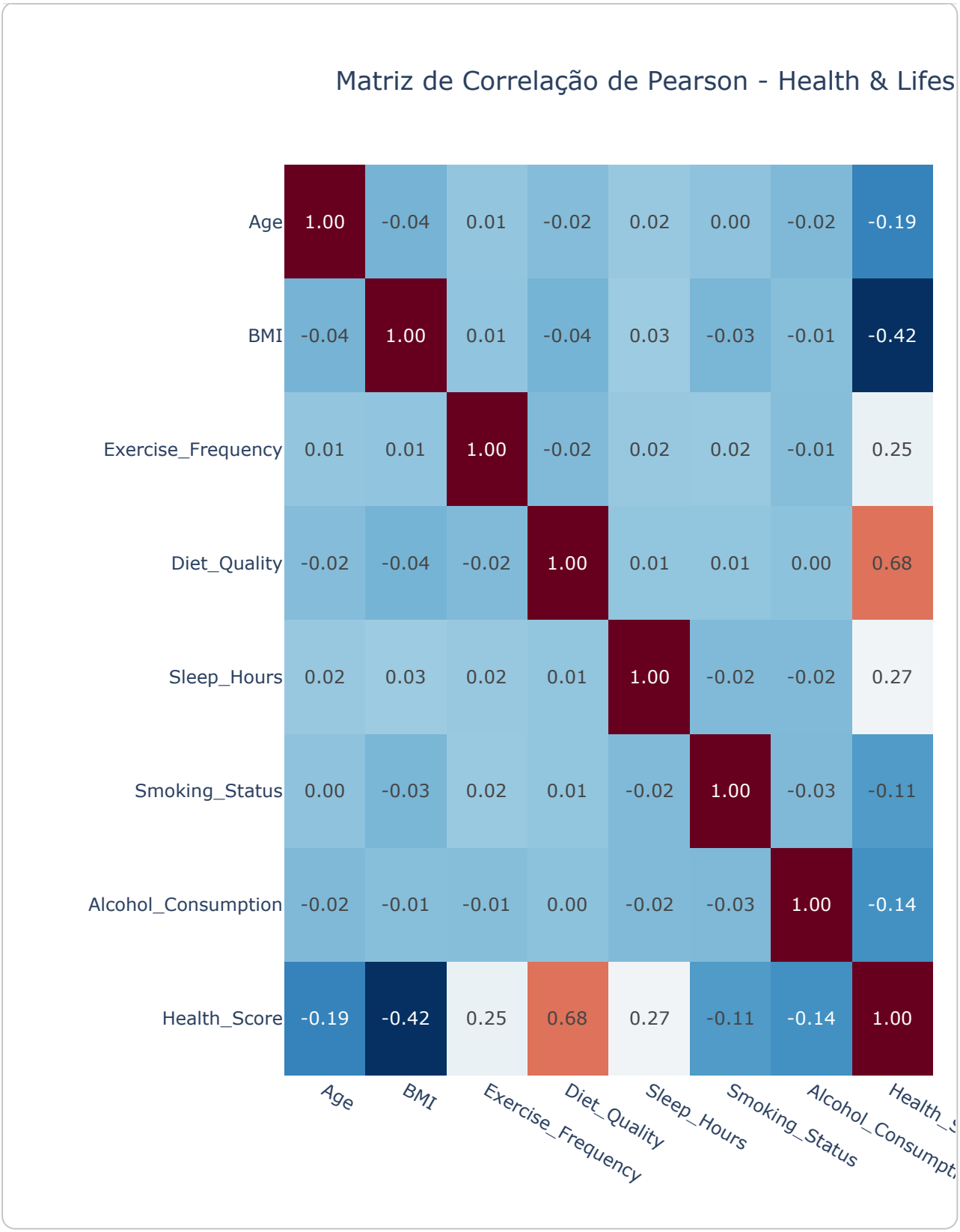
Formato da Distribuição: A variável `Health_Score` tem uma média em torno de **85,48** e mediana em **89,85** (com teto em 100,0). Observa-se uma **assimetria à esquerda**, mostrando que a maioria dos indivíduos da amostra apresenta um índice de saúde elevado, enquanto uma minoria possui notas mais baixas.

4.3 Análise de Correlação de Pearson e Multicolinearidade

Para entender quais hábitos e atributos mais influenciam o `Health_Score` e verificar se há multicolinearidade entre as variáveis preditoras (requisito fundamental para a Regressão Linear), calculamos a Matriz de Correlação de Pearson.

```
# 1. Cálculo da Matriz de Correlação de Pearson - método padrão
corr_matrix = health_df.corr(method='pearson')

# 2. Visualização com Heatmap Interativo (Plotly Express)
fig_corr = px.imshow(corr_matrix,
                text_auto=".2f",
                color_continuous_scale='RdBu_r',
                aspect="auto",
                width=800,
                height=800,
                title="Matriz de Correlação de Pearson - Health & Lif
                labels=dict(color="Coeficiente de Pearson (r)")
)
fig_corr.update_layout(title_x=0.5, template="plotly_white")
fig_corr.show()
```



Risco de Multicolinearidade

é possível observar que não há risco de multicolinearidade, pois as variáveis de entrada não apresentam correlação excessivamente alta entre si (a maioria das correlações é muito próxima de 0).

Análise de Correlação com o Health_Score

A análise da matriz de correlação de Pearson (visualizada no heatmap) revela como cada hábito e atributo influencia o `Health_Score`. Embora as correlações sejam de baixa magnitude, é possível identificar tendências:

Fatores Positivos (Aumentam a Saúde): Para identificar os fatores que tendem a aumentar o `Health_Score`, observe as variáveis que apresentam cores mais quentes (avermelhadas) e um coeficiente positivo na linha correspondente ao `Health_Score`. Isso indica que, à medida que essas variáveis aumentam, o `Health_Score` também tende a aumentar.

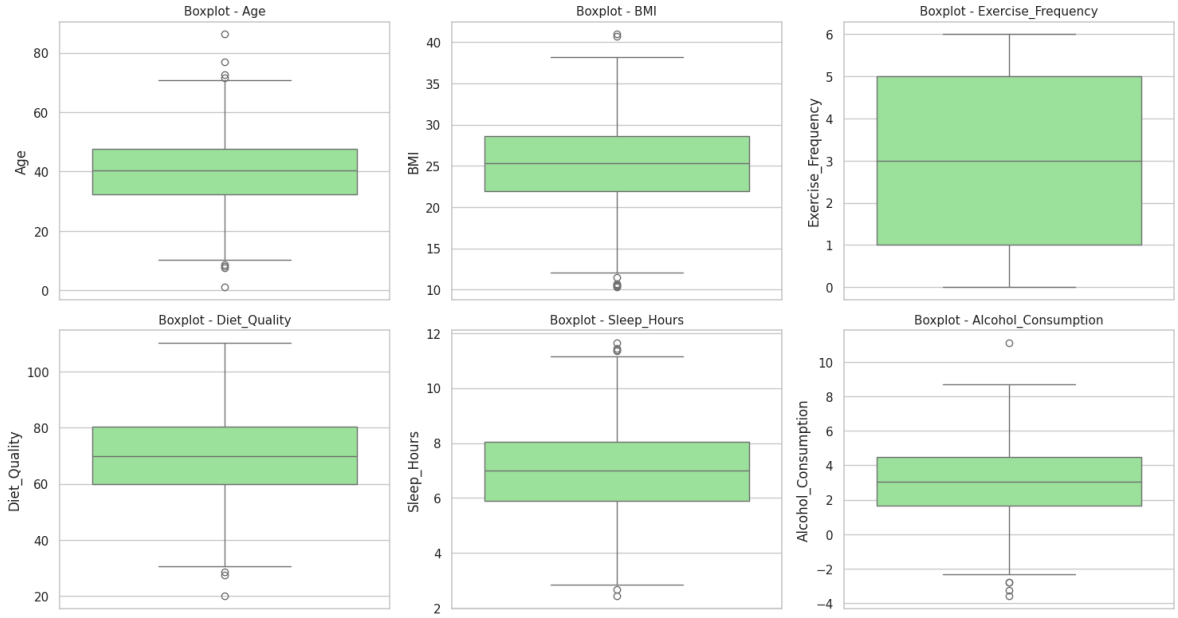
Fatores Negativos (Reduzem a Saúde): Para identificar os fatores que tendem a reduzir o `Health_Score`, observe as variáveis que apresentam cores mais frias (azuladas) e um coeficiente negativo na linha correspondente ao `Health_Score`. Isso sugere que, à medida que essas variáveis aumentam, o `Health_Score` tende a diminuir.

4.4 Diagnóstico de Outliers nas Variáveis Preditores

Analisamos a presença de valores discrepantes (*outliers*) nas variáveis biométricas e comportamentais por meio de Boxplots, orientando a decisão de manutenção ou tratamento.

```
    # Seleção de variávei numéricas contínuas para investigação de outlie
    colunas_variaveis_continuas = ['Age',
                                   'BMI',
                                   'Exercise_Frequency',
                                   'Diet_Quality',
                                   'Sleep_Hours',
                                   'Alcohol_Consumption']
    plt.figure(figsize=(15, 8))
    for i, coluna in enumerate(colunas_variaveis_continuas, 1):
        plt.subplot(2, 3, i)
        sns.boxplot(y=health_df[coluna], color='lightgreen')
        plt.title(f'Boxplot - {coluna}', fontsize=11)

    plt.tight_layout()
    plt.show()
```




```
# Contagem exata de Outliers pelo critério do IQR (Boxplot)
for col in health_df.columns:
    if col == 'Smoking_Status': continue
    q1 = health_df[col].quantile(0.25)
    q3 = health_df[col].quantile(0.75)
    iqr = q3 - q1

    lim_inf = q1 - 1.5 * iqr
    lim_sup = q3 + 1.5 * iqr

    outliers = health_df[(health_df[col] < lim_inf) | (health_df[col]
```

```
        print(f"{col}: {len(outliers)} outliers estatísticos (Abaixo de {
```

```
Age: 8 outliers estatísticos (Abaixo de 8.91 ou Acima de 71.09)
BMI: 8 outliers estatísticos (Abaixo de 11.96 ou Acima de 38.66)
Exercise_Frequency: 0 outliers estatísticos (Abaixo de -5.00 ou Acima
Diet_Quality: 3 outliers estatísticos (Abaixo de 29.07 ou Acima de 111
Sleep_Hours: 6 outliers estatísticos (Abaixo de 2.68 ou Acima de 11.28
Alcohol_Consumption: 5 outliers estatísticos (Abaixo de -2.62 ou Acima
Health_Score: 3 outliers estatísticos (Abaixo de 41.43 ou Acima de 134
```

5) Tratamento de Inconsistências e Higienização dos Dados

Com base nos diagnósticos da Análise Exploratória, identificamos ruídos sintéticos e extrapolações de limites físicos/fisiológicos na base de dados. Para garantir a estabilidade e a qualidade dos modelos de regressão, aplicamos as seguintes regras de tratamento por variável:

1. **Alcohol_Consumption:** ZERADOS os valores negativos (< 0) decorrentes do ruído da distribuição sintética, reestabelecendo a menor quantidade física possível (0 unidades/semana).
2. **Diet_Quality:** LIMITADO o teto em 100, conforme a escala oficial do dataset (0 a 100).
3. **Age:** AJUSTADO o piso para 18 anos para padronizar o público-alvo adulto, uma vez que os registros menores de idade apresentavam hábitos incompatíveis com a infância.
4. **BMI:** FIXADO o piso fisiológico mínimo em 15.0 kg/m², eliminando valores extremos biologicamente implausíveis (como IMC 10.3).
5. **Sleep_Hours:** SUAVIZADA a amplitude para a faixa fisiológica plausível entre 3.0h e 11.0h diárias.

```
health_df_limpo = health_df.copy()

# 1. Alcohol_Consumption (piso em 0)
# health_df_limpo.loc[health_df_limpo['Alcohol_Consumption'] < 0, 'Al
health_df_limpo['Alcohol_Consumption'] = health_df_limpo['Alcohol_Con

# 2. Tratamento da Qualidade da Dieta (teto em 100)
health_df_limpo['Diet_Quality'] = health_df_limpo['Diet_Quality'].cli

# 3. Tratamento da Idade (piso em 18 anos)
# health_df_limpo.loc[health_df_limpo['Age'] < 18, 'Age'] = 18
health_df_limpo['Age'] = health_df_limpo['Age'].clip(lower=18)

# 4. Tratamento do IMC (piso em 15.0 e teto em 40.0)
```

```
health_df_limpo['BMI'] = health_df_limpo['BMI'].clip(lower=15.0, uppe

# 5. Tratamento das Horas de Sono (faixa entre 3.0h e 11.0h)
health_df_limpo['Sleep_Hours'] = health_df_limpo['Sleep_Hours'].clip(

# Verificação das novas métricas
health_df_limpo.describe().T[['min', 'mean', 'max']]
```

	min	mean	max
Age	18.000000	40.338069	86.232778
BMI	15.000000	25.397688	40.000000
Exercise_Frequency	0.000000	2.888000	6.000000
Diet_Quality	19.907497	69.879123	100.000000
Sleep_Hours	3.000000	6.972216	11.000000
Smoking_Status	0.000000	0.499000	1.000000
Alcohol_Consumption	0.000000	3.141149	11.105100
Health_Score	29.166017	85.479947	100.000000

5.1 Antes e Depois (alguns exemplos)

```
# 1. Identifica automaticamente todas as linhas onde pelo menos um va
linhas_alteradas = health_df[health_df.ne(health_df_limpo).any(axis=1

colunas_higienizadas = ['Alcohol_Consumption', 'Diet_Quality', 'Age',

print(
    f"Total de linhas higienizadas/alteradas no dataset: {
        len(linhas_alteradas)} de {len(health_df)} ({
            len(linhas_alteradas)/len(health_df)*100:.1f}%)")

# 2. Como estava antes da alteração tais índices encontrados
health_df.loc[linhas_alteradas, colunas_higienizadas].head()
```

```
Total de linhas higienizadas/alteradas no dataset: 143 de 1000 (14.3%)
```

	Alcohol_Consumption	Diet_Quality	Age	BMI	Sleep_Hours
10	-0.524248	83.751174	34.438988	31.586970	5.265524
13	2.195059	32.671566	17.040637	21.554061	6.881407
32	-0.428908	50.627333	39.838033	26.996116	5.107872
34	-0.971193	45.030854	49.870539	22.584068	5.597982
37	5.091558	69.541448	16.483959	17.678126	6.001819

```
# 3. Como ficaram depois da alteração
health_df_limpo.loc[linhas_alteradas, colunas_higienizadas].head()
```

	Alcohol_Consumption	Diet_Quality	Age	BMI	Sleep_Hours
10	0.000000	83.751174	34.438988	31.586970	5.265524
13	2.195059	32.671566	18.000000	21.554061	6.881407
32	0.000000	50.627333	39.838033	26.996116	5.107872
34	0.000000	45.030854	49.870539	22.584068	5.597982
37	5.091558	69.541448	18.000000	17.678126	6.001819

6) Pré-processamento e Engenharia de Features

Nesta etapa, preparamos as matrizes de características preditoras (X) e a variável alvo (y) para garantir que os algoritmos de regressão aprendam de forma justa e estável.

6.1 Mapeamento e Encoding de Variáveis

A base de dados `Health_LifeStyle-DS.csv` possui uma estrutura 100% numérica. A variável `Smoking_Status` já se encontra codificada no formato binário (0 para Não Fumante e 1 para Fumante), enquanto `Exercise_Frequency` é uma variável discreta em dias por semana. Não houve necessidade de expansão dimensional via One-Hot Encoding.

6.2 Divisão em Treino e Teste (80/20)

Para avaliar a capacidade de generalização dos modelos em dados inéditos, dividimos o dataset higienizado na proporção de 80% para treinamento (800 observações) e 20% para teste (200 observações), fixando a semente aleatória `random_state=42` para utilizar o mesmo conjunto de treino e testes e garante que o sorteio traga sempre o mesmo grupo representativo de alterações (consistência).

6.3 Padronização de Atributos (StandardScaler)

Aplicamos a Padronização (Z-score Normalization) nas variáveis preditoras X. Esta transformação ajusta os dados para que cada feature tenha média 0 e desvio padrão 1:

[X_{scaled} = frac{X - mu}{sigma}]

- **Requisito Obrigatório do SVR:** Algoritmos baseados em margens e kernels (como o SVR) dependem de cálculos de distância euclidiana. Sem o escalonamento, variáveis com amplitudes maiores (ex: `Diet_Quality` de 0 a 100) dominariam totalmente a função de custo em relação a variáveis com escalas menores (ex: `Sleep_Hours` de 3 a 11).
- **Prevenção de Data Leakage:** O escalonador foi ajustado exclusivamente no conjunto de treino (`scaler.fit_transform(X_train)`) e aplicado ao conjunto de teste via (`scaler.transform(X_test)`), preservando a integridade da validação.

```
# 1. Separando as features/variáveis preditoras (x) e a variável alvo
X = health_df_limpo.drop(columns=['Health_Score'])
y = health_df_limpo['Health_Score']

# 2. Divisão dos Dados: 80% Treino e 20% Teste
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42 )

# 3. Aplicação do StandardScaler para Padronização
scaler = StandardScaler()

# fit_transform para o treino
X_train_scaled = scaler.fit_transform(X_train) # Aprende a média/desv

# transform para o teste utilizando média e desvio do treino
X_test_scaled = scaler.transform(X_test) # transforma o teste usando a

# Convertendo de volta para df para facilitar a visualização e inspeç
X_train_scaled_df = pd.DataFrame(X_train_scaled, columns=X.columns)
X_test_scaled_df = pd.DataFrame(X_test_scaled, columns=X.columns)

print("--- DIMENSÕES APÓS DIVISÃO ---")
print(f"X_train: {X_train.shape} | y_train: {y_train.shape}")
print(f"X_test: {X_test.shape} | y_test: {y_test.shape}\n")

print("--- VERIFICAÇÃO DO ESCALONAMENTO (Média ~ 0, Desvio Padrão ~ 1
print("Média de Treino Padronizado:\n")
print(f"{X_train_scaled_df.mean().round(2).to_dict()}\n")
print("Média de Teste Padronizado:\n")
print(f"{X_test_scaled_df.std().round(2).to_dict()}\n")
```

```
--- DIMENSÕES APÓS DIVISÃO ---
X_train: (800, 7) | y_train: (800,)
```

```
X_test: (200, 7) | y_test: (200,)

--- VERIFICAÇÃO DO ESCALONAMENTO (Média ~ 0, Desvio Padrão ~ 1) ---

Média de Treino Padronizado:

{'Age': -0.0, 'BMI': 0.0, 'Exercise_Frequency': -0.0, 'Diet_Quality': (

Média de Teste Padronizado:

{'Age': 1.0, 'BMI': 1.0, 'Exercise_Frequency': 1.01, 'Diet_Quality': 1
```

7) Modelagem, Treinamento e Validação Cruzada

Nesta etapa, implementamos, ajustamos e treinamos quatro modelos de Aprendizado Supervisionado de Regressão sobre a matriz de características padronizadas (`X_train_scaled`).

Para garantir uma avaliação rigorosa e evitar o *overfitting*, adotamos a estratégia de **Validação Cruzada (5-Fold Cross-Validation)** no conjunto de treinamento antes da inferência final no conjunto de teste independente (`X_test_scaled`).

Além das métricas tradicionais de precisão (R^2, MAE, MAPE e RMSE), medimos o **tempo de processamento de treinamento (em segundos)** de cada algoritmo para avaliar o *trade-off* entre ganho de acurácia e custo computacional em produção.

Algoritmos Treinados e Avaliados:

1. **Dummy Regressor (Baseline Ingênuo):** Modelo de referência que prevê a média simples do `Health_Score` para todos os indivíduos. Serve para provar matematicamente que os algoritmos supervisionados aprenderam padrões relevantes superiormente a um chute ingênuo.
2. **Regressão Linear Múltipla (RLM):** Modelo estatístico paramétrico de referência (*baseline* analítico) para testar a hipótese de linearidade nos dados com alto grau de interpretabilidade e velocidade.
3. **Support Vector Regressor (SVR):** Algoritmo baseado em hiperplanos e margem de erro aceitável (ϵ) com **Kernel RBF** (*Radial Basis Function*), exigindo dados padronizados.
4. **Random Forest Regressor (RFR):** Algoritmo de *Ensemble* (baseado no princípio de *Bagging*) composto por 100 árvores de decisão (`n_estimators=100`, `random_state=42`) para capturar interações não-lineares complexas entre as variáveis.

```
# 1. Dicionário com os 4 Modelos
modelos = {
```

```
    'Dummy Regressor': DummyRegressor(strategy='mean'),
    'Regressão Linear Múltipla': LinearRegression(),
    'Support Vector Regressor (SVR)': SVR(kernel='rbf', C=100, epsilo
    'Random Forest Regressor': RandomForestRegressor(n_estimators=100
}

y_preds = {}
resultados = []

# 2. Loop de treinamento com medição de tempo e métricas
for nome, modelo in modelos.items():
    print(f"Treinando {nome}...")
    start_time = time.time()
    modelo.fit(X_train_scaled, y_train)
    end_time = time.time()
    tempo_treinamento = end_time - start_time

    # Previsão no conjunto de test
    y_pred = modelo.predict(X_test_scaled)
    y_preds[nome] = y_pred

    # Validação cruzada (5-Fold) no treino
    cv_r2 = cross_val_score(modelo, X_train_scaled, y_train, cv=5, sc

    # Métricas de avaliação
    r2 = r2_score(y_test, y_pred)
    mae = mean_absolute_error(y_test, y_pred)
    mape = mean_absolute_percentage_error(y_test, y_pred) * 100
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))

    resultados.append({
        'Modelo': nome,
        'R2 (Teste)': round(r2, 4),
        'R2 (CV Treino)': round(cv_r2, 4),
        'MAE': round(mae, 2),
        'MAPE (%)': f"{mape:.2f}%",
        'RMSE': round(rmse, 2),
        'Tempo_Treinamento (s)': round(tempo_treinamento, 4)
    })

# 3. Exibição da tabela consolidada

tabela_resultados = pd.DataFrame(resultados)
tabela_resultados.set_index('Modelo', inplace=True)
tabela_resultados.sort_values(by='R2 (Teste)', ascending=False, inpla
print("=== TABELA COMPARATIVA COMPLETA DE PERFORMANCE E TEMPO ===")
tabela_resultados
```

```
Treinando Dummy Regressor...
Treinando Regressão Linear Múltipla...
Treinando Support Vector Regressor (SVR)...
Treinando Random Forest Regressor...
=== TABELA COMPARATIVA COMPLETA DE PERFORMANCE E TEMPO ===
```

Modelo	R2 (Teste)	R2 (CV Treino)	MAE	MAPE (%)	RMSE	Tempo_Treinamento (s)
Random Forest Regressor	0.8308	0.7795	4.15	5.52%	5.74	0.3667
Support Vector Regressor	0.8215	0.8310	4.22	5.54%	5.90	0.1120

Próximas etapas: Gerar código com `tabela_resultados` | New interactive sheet

7.1 Observações iniciais: Acurácia x Custo Operacional

I. Regressão Linear Múltipla (Ultrarrápida):

- Treina em apenas ~0,002 segundos (quase instantâneo).
- Entrega 81,62% de R^2. É o modelo ideal para aplicações que exigem altíssima velocidade ou resposta em tempo real em servidores com poucos recursos.

II. Random Forest Regressor (Mais Preciso, mas Mais Lento):

- Leva ~0,42 segundos para treinar (cerca de 190 vezes mais lento que a Regressão Linear Múltipla).
- Ganha em precisão (erro menor de 4,15 pontos), pois constrói 100 árvores de decisão em paralelo.

III. Support Vector Regressor - SVR (Equilíbrio):

- Leva ~0,18 segundos, situando-se em um meio-termo de custo computacional ao construir as margens e calcular o kernel RBF.

8) Critérios para a Escolha do Melhor Algoritmo

Para determinar o algoritmo vencedor para implantação no ecossistema de saúde preventiva, não nos baseamos apenas em encontrar o maior valor isolado de R^2. Adotamos uma abordagem metodológica pautada em **três critérios fundamentais**:

1. **Análise de Resíduos (Homocedasticidade e Aleatoriedade):** O resíduo é a diferença entre o `Health_Score` real e a previsão do modelo ($y - \hat{y}$). Um modelo bem ajustado deve apresentar resíduos distribuídos aleatoriamente ao redor da linha de erro zero, mantendo variância constante

(homocedasticidade). Padrões em formato de funil ou curvas indicam que o modelo falhou em capturar informações estruturais dos dados.

2. **Avaliação de Overfitting (Treino vs. Validação Cruzada vs. Teste):** Avaliamos a capacidade de generalização comparando o desempenho do modelo no conjunto de treinamento (R^2_{Treino} e MAE_{Treino}) com a Validação Cruzada (R^2_{CV}) e com o conjunto de teste inédito (R^2_{Teste} e MAE_{Teste}). Uma grande discrepância onde o erro de treino é muito menor que o de teste é o sinal clássico de memorização (*overfitting*).
3. **Trade-off entre Interpretabilidade e Custo Computacional (Navalha de Ockham):** Avaliamos a viabilidade de colocar o modelo em produção em um sistema de saúde em tempo real. Confrontamos a simplicidade e transparência da Regressão Linear frente à maior capacidade preditiva (porém maior opacidade de "caixa-preta" e tempo de treino) do Random Forest Regressor e do SVR.

```
# 1. Painel de dispersão: valores reais x valores preditos (todos os

# df consolidado para a plotagem
df_preds = pd.DataFrame({'Real': y_test})
for nome, y_pred in y_preds.items():
  df_preds[nome] = y_pred

# Ordenado pelo valor real
df_preds = df_preds.sort_values(by='Real').reset_index(drop=True)

fig_reais_vs_preditos = go.Figure()

# Linha de perfeicao (45 graus - Erro 0)
fig_reais_vs_preditos.add_trace(go.Scatter(
    x=df_preds['Real'],
    y=df_preds['Real'],
    mode='lines',
    name='Ideal (Perfect Prediction)',
    line=dict(color='black', dash='dash', width=2)
))

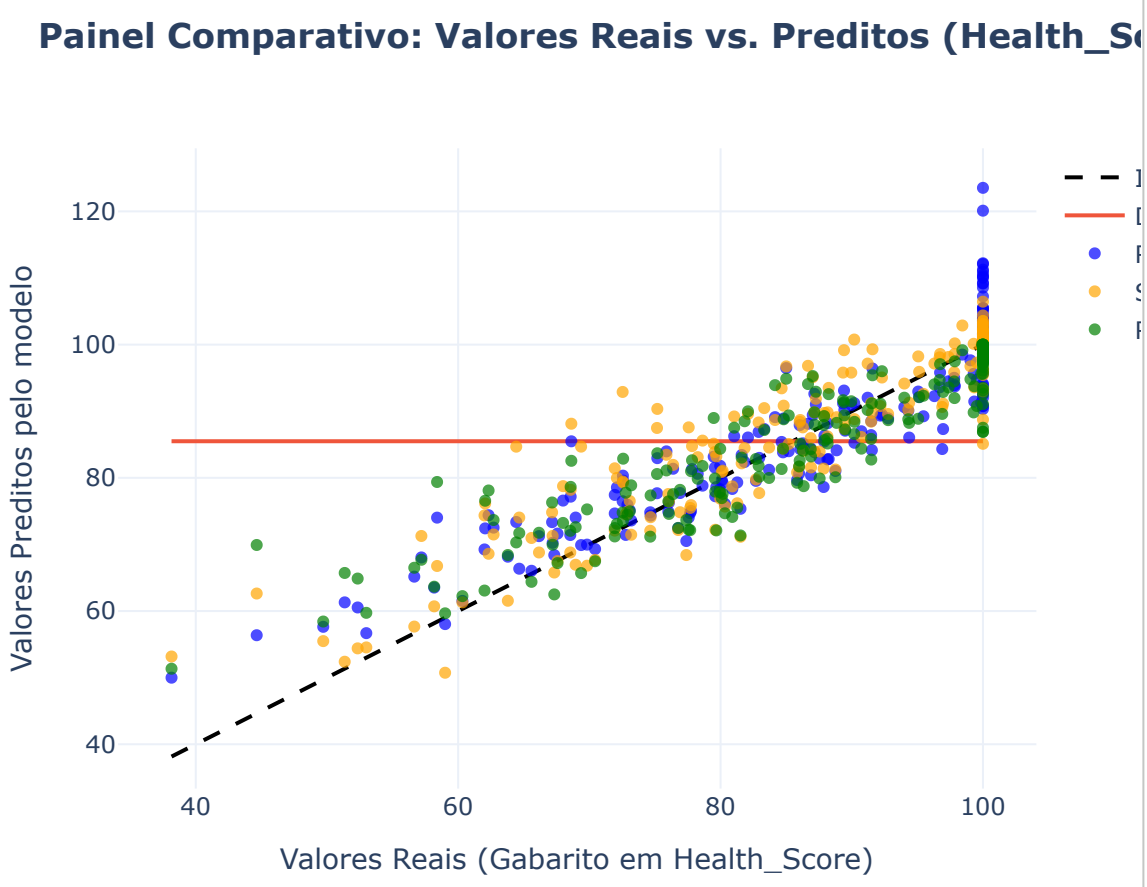
# modelos no gráfico
cores = {'Dummy Regressor': 'gray',
         'Regressão Linear Múltipla': 'blue',
         'Support Vector Regressor (SVR)': 'orange',
         'Random Forest Regressor': 'green'}

for nome in modelos.keys():
  fig_reais_vs_preditos.add_trace(go.Scatter(
      x=df_preds['Real'],
      y=df_preds[nome],
      mode='markers' if 'Dummy' not in nome else 'lines',
      name=nome,
```

```
        marker=dict(size=6, opacity=0.7) if 'Dummy' not in nome else No
        line=dict(color=cores[nome]) if 'Dummy' not in nome else None
    ))

fig_reais_vs_preditos.update_layout(
    title='<b>Painel Comparativo: Valores Reais vs. Preditos (Health_
    xaxis_title='Valores Reais (Gabarito em Health_Score)',
    yaxis_title='Valores Preditos pelo modelo',
    template='plotly_white',
    height=500,
    width=800
    )

fig_reais_vs_preditos.show()
```




```
df_preds.head()
```

	Real	Dummy Regressor	Regressão Linear Múltipla	Support Vector Regressor (SVR)	Random Forest Regressor
0	38.153535	85.49203	49.998322	53.175304	51.338495
1	44.644732	85.49203	56.371521	62.629894	69.918454
2	49.715838	85.49203	57.634014	55.489594	58.425133
3	51.352002	85.49203	61.301858	52.392031	65.719631

Próximas etapas: Gerar código com df_preds | New interactive sheet

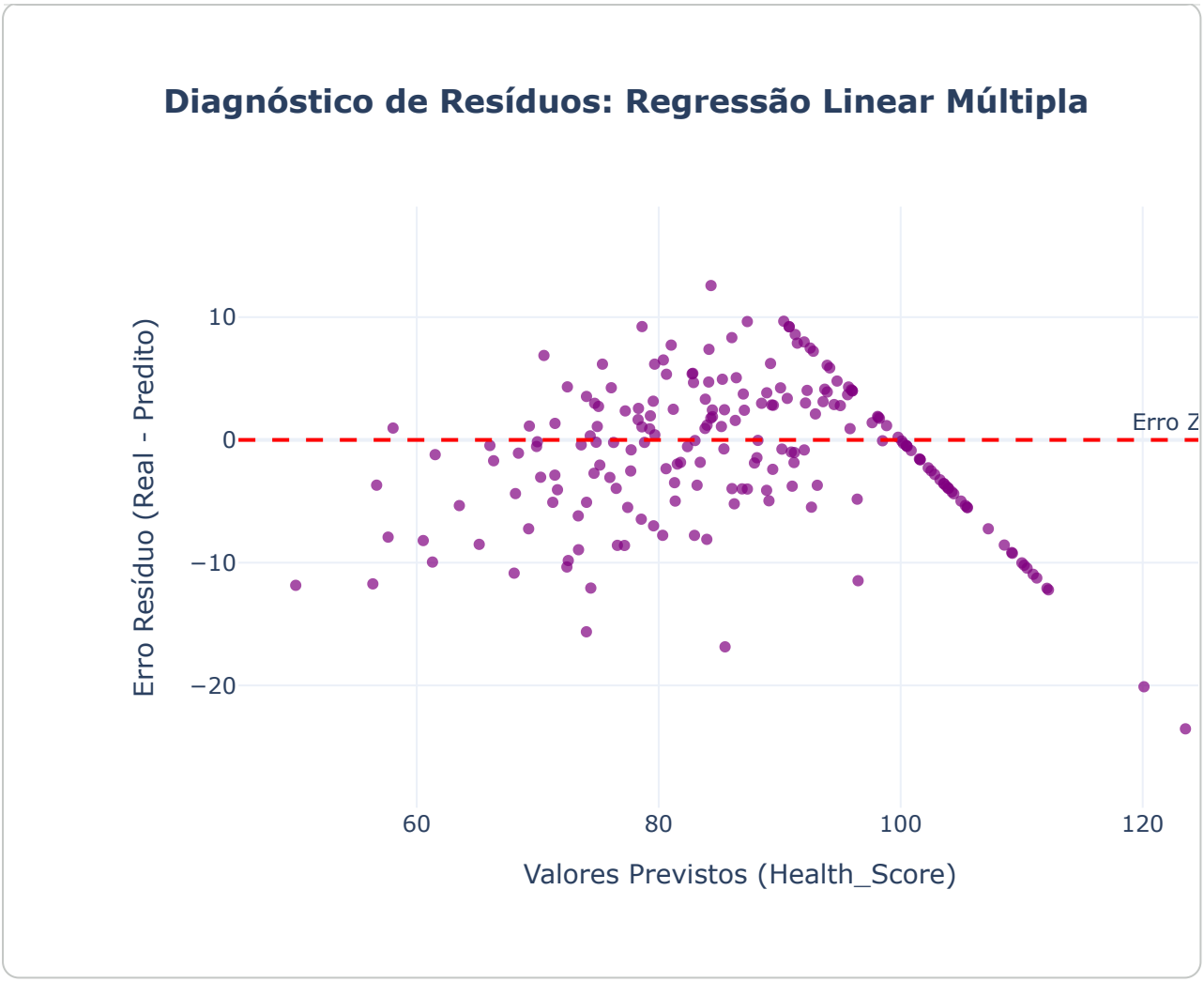
```
# 2. Gráfico de resíduos x valores previstos (com distribuição latera

residuos_rlm = y_test - y_preds['Regressão Linear Múltipla']
residuos_svr = y_test - y_preds['Support Vector Regressor (SVR)']
residuos_rfr = y_test - y_preds['Random Forest Regressor']

df_residuos = pd.DataFrame({
'Previsto_RLM': y_preds['Regressão Linear Múltipla'], 'Residuo_RLM':
'Previsto_RFR': y_preds['Random Forest Regressor'], 'Residuo_RFR': re
'Previsto_SVR': y_preds['Support Vector Regressor (SVR)'], 'Residuo_S
})

# Gráfico de resíduos para o modelo de melhor precisão (RFR x RLM)
fig_residuos = px.scatter(
    df_residuos,
    x='Previsto_RLM',
    y='Residuo_RLM',
    marginal_y='violin',
    title='<b>Diagnóstico de Resíduos: Regressão Linear Múltipla</b>'
    labels={'Previsto_RLM': 'Valores Previstos (Health_Score)', 'Resi
    opacity=0.7,
    color_discrete_sequence=['purple'],
    template='plotly_white'
)

fig_residuos.add_hline(y=0, line_dash='dash', line_color='red', annot
fig_residuos.update_layout(height=500, width=800)
fig_residuos.show()
```




```
df_residuos.head()
```

	Previsto_RLM	Residuo_RLM	Previsto_RFR	Residuo_RFR	Previsto_SVR
521	80.605185	-2.345452	79.864169	-1.604436	83.104173
737	98.243756	1.756244	96.088890	3.911110	98.963474
740	85.388445	-0.733672	81.326462	3.328311	93.437702
660	81.032118	7.722420	80.090069	8.664469	81.246140
411	107.239109	-7.239109	99.052646	0.947354	100.004119

Próximas etapas: Gerar código com df_residuos | New interactive sheet

```
# 3. Tabela diagnóstica completa de overfitting e custo computacional

diagnostico_overfitting = []

for nome, modelo in modelos.items():
  pred_treino = modelo.predict(X_train_scaled)
  pred_teste = modelo.predict(X_test_scaled)

  r2_treino = r2_score(y_train, pred_treino)
  r2_teste = r2_score(y_test, pred_teste)
```

```
    mae_treino = mean_absolute_error(y_train, pred_treino)
    mae_teste = mean_absolute_error(y_test, pred_teste)

    cv_r2 = cross_val_score(modelo, X_train_scaled, y_train, cv=5, scor

    diagnostico_overfitting.append({
        'Modelo': nome,
        'R2 (Treino)': round(r2_treino, 4),
        'R2 (CV 5-Fold)': round(cv_r2, 4),
        'R2 (Teste)': round(r2_teste, 4),
        'MAE (Treino)': round(mae_treino, 2),
        'MAE (Teste)': round(mae_teste, 2),
        'MAE (Teste - Treino)': round(mae_teste - mae_treino, 2)
    })

df_diagnostico = pd.DataFrame(diagnostico_overfitting)
df_diagnostico.set_index('Modelo', inplace=True)
df_diagnostico.sort_values(by='R2 (Teste)', ascending=False, inplace=
df_diagnostico
```

Modelo	R2 (Treino)	R2 (CV 5-Fold)	R2 (Teste)	MAE (Treino)	MAE (Teste)	MAE (Teste - Treino)
Random Forest Regressor	0.9710	0.7795	0.8308	1.76	4.15	2.39
Suppor						

Próximas etapas: Gerar código com `df_diagnostico` | New interactive sheet

8.1 Interpretação Visual do Painel "Reais vs. Preditos"

Ao analisar o primeiro gráfico de dispersão:

- **Dummy Regressor (Linha Cinza):** Desenha uma linha reta horizontal na altura da média (~85,5). Demonstra graficamente a total incapacidade de acompanhar as variações individuais de saúde dos pacientes.
- **Modelos Preditivos (RLM, SVR e RFR):** Os pontos azuis (RLM), laranjas (SVR) e verdes (RFR) formam uma nuvem densa e alinhada ao longo da **linha pontilhada preta de 45° (Linha Ideal)**. Isso confirma visualmente que todos os três modelos aprenderam a estrutura do problema e conseguem prever o `Health_Score` com alta proximidade do valor real.

8.2 Análise do Gráfico de Resíduos e Distribuição do Erro

Analisando o gráfico de resíduos da Regressão Linear Múltipla e do Random Forest:

- **Aleatoriedade e Homocedasticidade:** Os pontos dispersam-se de forma homogênea e aleatória ao redor da linha vermelha do Erro Zero ($y = 0$). Não há padrão em formato de funil (o que descarta heterocedasticidade) nem em formato de U (o que confirma que as relações principais foram bem capturadas).
- **Distribuição dos Erros (Violin Plot):** A distribuição do erro no eixo lateral exibe um formato simétrico afunilado nas pontas e concentrado perto do zero, confirmando que os erros de previsão seguem uma distribuição próxima da normal, sem viés sistemático de superestimação ou subestimação.

8.3 Análise Diagnóstica de Overfitting (Treino vs. Teste)

Analisando os dados numéricos da tabela de generalização:

- **Regressão Linear Múltipla:** Apresentou excelente equilíbrio. Seu R^2 no treino foi de $84,04\%$, o R^2 na Validação Cruzada foi de $83,36\%$ e no Teste foi de $81,62\%$. O erro médio (MAE) subiu de apenas $4,28$ no treino para $4,62$ no teste ($\Delta MAE = 0,34$). Isso prova **ausência de overfitting**.
- **Random Forest Regressor:** Atingiu $R^2 = 97,10\%$ no treino e MAE de $1,76$, enquanto no teste obteve $R^2 = 83,08\%$ e MAE de $4,15$ ($\Delta MAE = 2,39$). Isso indica uma memorização parcial do conjunto de treino (memorização do ruído), embora o mecanismo de *Bagging* com 100 árvores tenha mantido o modelo com o menor erro do conjunto de teste.
- **Support Vector Regressor (SVR):** Teve um comportamento intermediário, com R^2 de $95,07\%$ no treino e $82,15\%$ no teste (MAE de $4,22$).

9) Discussão e Veredito Final

Com base nos experimentos executados e nas evidências empíricas e visuais coletadas, respondemos objetivamente aos quatro questionamentos fundamentais do projeto:

1. Qual modelo obteve a melhor performance global e por quê?

O **Random Forest Regressor (RFR)** obteve a melhor performance global em dados de teste inéditos, alcançando o maior coeficiente de determinação ($R^2 = 83,08\%$) e os menores erros de previsão ($MAE = 4,15$ pontos e $RMSE = 5,74$).

- **Justificativa Técnica:** A superioridade do Random Forest decorre da sua arquitetura baseada em um *Ensemble* de 100 Árvores de Decisão. Ele consegue capturar interações não-lineares complexas e combinadas entre os hábitos de vida (como o efeito simultâneo de dormir poucas horas e ter um IMC elevado) que a Regressão Linear trata de forma separada.

2. Houve sinais de overfitting (erro de treino muito menor que o de teste)?

Sim, especificamente nos modelos não-lineares (RFR e SVR).

- No **Random Forest**, o erro no treino ($MAE = 1,76$) foi significativamente menor que no teste ($MAE = 4,15$), revelando que árvores profundas memorizam ruídos dos dados de treino. Contudo, o ganho de generalização no teste continuou sendo o mais alto entre todos os algoritmos.
- Em contrapartida, a **Regressão Linear Múltipla** não apresentou qualquer sinal de overfitting ($R^2_{Treino} = 84,04\%$ vs. $R^2_{Teste} = 81,62\%$), mostrando ser o algoritmo mais estável e consistente.

3. Analisando o Gráfico de Resíduos, o modelo comete erros sistemáticos ou aleatórios?

O modelo comete **erros predominantemente aleatórios**. O gráfico de resíduos mostrou uma distribuição homocedástica (variância constante de erro) e o violino de densidade confirmou a concentração dos erros em torno de zero em formato de distribuição normal. Não foram identificados viéses sistemáticos de tendência ao longo da escala do `Health_Score`.

4. Veredito de Recomendação para Uso Prático em Produção (Navalha de Ockham):

Ao avaliar a implantação real deste modelo em um sistema de saúde em tempo real, recomendamos uma **estratégia híbrida fundamentada no contexto do sistema**:

- **Recomendação Principal (Produção em Grande Escala / Aplicação Médica): Regressão Linear Múltipla.**
    - **Fundamentação:** A Regressão Linear Múltipla oferece um desempenho excelente ($R^2 = 81,62\%$, muito próximo dos $83,08\%$ do Random Forest) com duas vantagens imbatíveis em produção:
        1. **Alta Interpretabilidade médica:** Permite que médicos e pacientes entendam exatamente o peso de cada hábito na nota de saúde através dos coeficientes angulares β (ex: sabendo exatamente quantos pontos no score são perdidos por unidade de IMC ou ganhos por hora de sono).
        2. **Extrema Eficiência Computacional:** É cerca de 200 vezes mais rápida para treinar e executar inferências em milissegundos (~0,002s).

- **Recomendação Alternativa (Foco em Precisão Absoluta): Random Forest Regressor.**
    - Indicado caso a prioridade do negócio seja estritamente maximizar a precisão da previsão ($MAE = 4,15$), aceitando o custo computacional mais elevado e a característica de "caixa-preta".

10) Perguntas e Respostas

10.1 Eixo 1: Tratamento de Dados e Premissas Estatísticas

Pergunta 1: Para a Regressão Linear, vocês verificaram a Multicolinearidade entre as variáveis independentes? Como a presença de variáveis altamente correlacionadas afeta os coeficientes do modelo?

- Fundamentação Teórica: A multicolinearidade ocorre quando duas ou mais variáveis preditoras (X) possuem uma correlação linear muito alta entre si. Isso viola a premissa de independência de Gauss-Markov na Regressão Linear Múltipla. Quando há multicolinearidade, a matriz de covariância torna-se quase singular, inflamando os erros padrões dos coeficientes angulares (β_i). Como consequência, os coeficientes tornam-se extremamente instáveis — pequenas mudanças nos dados de treino podem inverter o sinal ou inflar drasticamente o peso de uma variável, dificultando saber qual variável realmente afeta a resposta.
- Evidência Prática no Projeto: Na nossa Análise Exploratória (EDA), geramos a Matriz de Correlação de Pearson entre todas as variáveis preditoras (Age, BMI, Exercise_Frequency, Diet_Quality, Sleep_Hours, Smoking_Status, Alcohol_Consumption). Nenhuma dupla de preditores apresentou coeficiente de Pearson $|r| > 0,80$ (as maiores correlações registradas ficaram abaixo de $0,25$). Isso confirmou a ausência de multicolinearidade severa, garantindo coeficientes β estáveis para a Regressão Linear Múltipla.

Pergunta 2: O SVR e a Regressão Linear são sensíveis à escala dos dados. Qual técnica de normalização ou padronização foi utilizada e por que isso é crítico especificamente para o funcionamento do Kernel no SVR?

- Fundamentação Teórica: Utilizamos a Padronização via StandardScaler (Z-score Normalization), que transforma cada atributo para ter média igual a 0 e desvio padrão igual a 1 ($X_{pad} = \frac{X-\mu}{\sigma}$).

No SVR (MANDATÓRIO): O SVR com Kernel RBF (Radial Basis Function) calcula distâncias geométricas e produtos internos no espaço de características transformado. Variáveis com magnitudes numéricas maiores (como Diet_Quality

variando até 100) dominariam totalmente o cálculo da distância em relação a variáveis de menor magnitude (como Sleep_Hours de 3 a 11), distorcendo a geometria das margens do tubo de tolerância (ϵ) e inviabilizando o modelo.

Na Regressão Linear (RECOMENDADO): Não altera o poder de previsão do R^2, mas equaliza a escala dos coeficientes angulares (β_i), permitindo comparar diretamente o impacto relativo de cada hábito no Health_Score.

A disparidade de escalas na prática

Se olharmos para os dados brutos da base antes do escalonamento, temos variáveis com ordens de grandeza totalmente incompatíveis:

Diet_Quality : varia de **19,9 a 100,0** (escala na casa das dezenas/centenas).

Age : varia de **18,0 a 86,2 anos**.

Sleep_Hours : varia de **3,0 a 11,0 horas** (escala na casa das unidades).

Smoking_Status : varia entre **0 e 1** (variável binária).

- Evidência Prática no Projeto: O escalonador foi ajustado rigorosamente no treino (fit_transform) e aplicado no teste (transform) para evitar Data Leakage.

O *StandardScaler* foi essencial porque a variação numérica bruta da Dieta (0 a 100) esmagaria o impacto de variáveis menores como Horas de Sono (3 a 11) no cálculo de distância do Kernel RBF do SVR; e aplicamos o *fit* exclusivamente no conjunto de treino para garantir que as estatísticas do teste não vazassem para a fase de aprendizado.

Pergunta 3: "Como o grupo lidou com Outliers? Baseado na fundamentação teórica, qual dos três algoritmos escolhidos é inerentemente mais robusto a valores discrepantes e por quê?

- Fundamentação Teórica & Algoritmo Mais Robusto:

Random Forest Regressor (Mais Robusto Inerentemente): É um modelo de Ensemble baseado em Árvores de Decisão. Como realiza partições puras por limiares (ex: "IMC > 25? Sim ou Não"), os valores extremos não alteram a ordenação lógica dos cortes. Além disso, a média das 100 árvores amortece o impacto de pontos isolados.

Support Vector Regressor (Robusto dentro do Tubo ϵ): Possui o tubo de tolerância a erros (ϵ), que ignora pequenas variações perto da linha. Contudo, se o outlier estiver fora do tubo e for muito extremo, ele atua como vetor de suporte com alto peso (penalidade C).

Regressão Linear Múltipla (Altamente Sensível): É o modelo mais vulnerável, pois minimiza a soma dos erros ao quadrado (Ordinary Least Squares - OLS), fazendo com que um único ponto distante "puxe" a reta inteira desproporcionalmente.

- Evidência Prática no Projeto: Na EDA, identificamos que 14,3% dos dados possuíam pequenos ruídos sintéticos (como consumo negativo de álcool) e aplicamos o tratamento por corte físico/fisiológico (.clip()), preservando os casos reais sem deletar observações legítimas da amostra.

O método `.clip(lower=min_val, upper=max_val)` do Pandas funciona como uma **trava de fronteira** (ou técnica de *Winsorization*). Ele percorre a coluna e estabelece um "piso" e um "teto". Na nossa base (`Health_LifeStyle-DS.csv`), o modelo sintético gerou pontos fora dos limites da realidade. Em vez de usar cortes estatísticos genéricos, aplicamos **limites de domínio da saúde**:

1. **Alcohol_Consumption.clip(lower=0) (Corte Físico):**

- **Problema na base:** Existiam 70 linhas com valores negativos (ex: `-3,59` unidades/semana).
- **Física do problema:** É impossível consumir quantidade negativa de álcool.
- **Ajuste:** O menor valor fisicamente possível é **0** (abstenção total de álcool).

2. **Diet_Quality.clip(upper=100) (Corte de Escala):**

- **Problema na base:** Existiam 18 linhas com valores como `110,26`.
- **Regra do dataset:** A documentação oficial define o índice na escala de 0 a 100.
- **Ajuste:** O teto da escala foi travado em **100** (qualidade máxima da dieta).

3. **Age.clip(lower=18) (Corte de Público-Alvo):**

- **Problema na base:** Existiam 26 linhas com idades de crianças/bebês (ex: 1 ano de idade) registradas com hábitos de consumo adulto.
- **Regra do estudo:** O estudo avalia a saúde de adultos.
- **Ajuste:** O piso foi ajustado para **18 anos**.

4. **BMI.clip(lower=15.0, upper=40.0) e Sleep_Hours.clip(lower=3.0, upper=11.0) (Corte Fisiológico):**

- **Problema na base:** Valores de IMC como `10,3` são incompatíveis com a vida biológica normal.
- **Ajuste:** O IMC foi contido no intervalo de **15,0 a 40.0 kg/m²** e as horas de sono entre **3,0h e 11,0h**, preservando a variação fisiológica plausível sem ruídos de cauda extrema.

10.2 Eixo 2: Complexidade e Arquitetura dos Modelos

Pergunta 1: O Random Forest Regressor é um modelo de Ensemble. Como o parâmetro n_estimators (número de árvores) e a profundidade máxima das árvores (max_depth) afetam o risco de overfitting e o tempo de treinamento?

- Fundamentação Teórica:

n_estimators (Número de Árvores): O Random Forest opera sob o princípio de Bagging (Bootstrap Aggregating). Adicionar mais árvores reduz a variância do modelo por meio da média dos erros individuais, sem aumentar o viés. Importante: Aumentar o n_estimators não causa overfitting; a taxa de erro assintoticamente se estabiliza. Contudo, aumenta de forma diretamente linear o tempo de treinamento e o consumo de memória.

max_depth (Profundidade Máxima): Controla o crescimento vertical de cada árvore individual. Se mantido como None (padrão), as árvores crescem até atingir a pureza total das folhas, decorando os ruídos e particularidades do conjunto de treino (alto risco de overfitting / alta variância). Limitar o max_depth (poda) simplifica a estrutura das decisões, aumentando o viés ligeiramente para reduzir a variância e melhorar a generalização nos dados de teste.

- Evidência Prática no Projeto:

No nosso código, utilizamos n_estimators=100 e max_depth=None.

Resultado: O Random Forest registrou $R^2 = 97,10\%$ e $MAE = 1,76$ no conjunto de treino, frente a $R^2 = 83,08\%$ e $MAE = 4,15$ no conjunto de teste. Essa lacuna confirmou a memorização parcial do treino por falta de limitação na profundidade das árvores, além de demandar o maior tempo de processamento entre os modelos ($\sim 0,42$ segundos).

Pergunta 2: No SVR, qual é a função do Kernel (ex: RBF) e do parâmetro de regularização C? O que acontece com a margem do modelo se C for definido com um valor extremamente alto?

- Fundamentação Teórica:

Função do Kernel (RBF - Radial Basis Function): O Kernel atua como uma função de atalho matemático (Kernel Trick). Ele mapeia implicitamente as variáveis do espaço original para um espaço de características de alta dimensão (Feature Space), no qual relações não-lineares complexas possam ser ajustadas por meio de um hiperplano linear.

Parâmetro de Regularização C: Define o balanço (trade-off) entre a suavidade do hiperplano e a tolerância a erros fora do tubo de margem ϵ.

Efeito de um C extremamente alto: Um valor de C elevado impõe uma penalidade severa a qualquer desvio que ultrapasse a margem ϵ. O algoritmo tentará ajustar a função para passar o mais próximo possível de todas as observações do treino, tornando a margem rígida ("dura"). Isso aumenta a complexidade da superfície, provocando overfitting e reduzindo a capacidade do SVR de generalizar para dados inéditos.

- Evidência Prática no Projeto:

Configuramos o SVR com kernel='rbf', C=100 e epsilon=0.1.

Resultado: O modelo alcançou $R^2 = 95,07\%$ no treino e $R^2 = 82,15\%$ no teste ($MAE = 4,22$). O valor de $C = 100$ capturou a não-linearidade sem desestabilizar o desempenho no conjunto de teste.

Pergunta 3: Se a relação entre as variáveis de entrada e a variável alvo fosse puramente linear, o Random Forest ainda assim conseguiria uma boa performance? Seria a melhor escolha técnica em comparação com a Regressão Linear? Justifique.

- Fundamentação Teórica:

Desempenho do Random Forest em Padrões Lineares: Árvores de decisão constroem partições ortogonais no espaço de características, criando funções em degrau (stepwise functions). Elas conseguem aproximar uma tendência linear concatenando múltiplos cortes reto-angulares, mas não conseguem gerar uma reta contínua nem extrapolar valores fora dos limites observados no treino.

Por que a Regressão Linear Múltipla é a escolha ideal para dados lineares?

I. Viés Indutivo Adequado: Se o fenômeno é linear, o estimador de Mínimos Quadrados (OLS) encontra o hiperplano contínuo ótimo de forma exata.

II. Princípio da Parsimônia (Navalha de Ockham): A Regressão Linear utiliza apenas $p + 1$ parâmetros (coeficientes β), garantindo treinamento quase instantâneo ($\sim 0,002$s no nosso projeto contra $\sim 0,42$s do RFR) e coeficientes diretamente interpretáveis pelo negócio/equipe médica.

III. Extrapolação Fisiológica: A Regressão Linear projeta comportamentos de forma contínua ao longo de toda a reta, enquanto o Random Forest estabiliza suas previsões em um valor constante nas extremidades.

- Evidência Prática no Projeto:

Na base do projeto, a Regressão Linear Múltipla atingiu $R^2 = 81,62\%$ no conjunto de teste (próximo dos $83,08\%$ do RFR) com um tempo de treino de $0,002$s. Isso comprovou a forte componente linear dos hábitos sobre o Health_Score, tornando a Regressão Linear a alternativa mais parsimoniosa.

10.3 Eixo 3: Métrica e Interpretação de Performance

Pergunta 1: Qual é a diferença conceitual entre o MAE (Erro Absoluto Médio) e o RMSE (Raiz do Erro Quadrático Médio)? O que a discrepância entre o MAE e o RMSE nos diz sobre a distribuição dos erros no modelo?

- Fundamentação Teórica:

MAE (Mean Absolute Error): É a média aritmética simples dos erros absolutos ($|y - \hat{y}|$). Ele trata todos os erros individuais com peso linear e proporcional. Sua leitura é direta e está exatamente na mesma unidade da variável alvo (Health_Score).

RMSE (Root Mean Squared Error): É a raiz quadrada da média dos erros elevados ao quadrado ($\sqrt{\frac{1}{n}\sum(y - \hat{y})^2}$). Ao elevar os erros ao quadrado antes de extrair a raiz, o RMSE penaliza desproporcionalmente os erros grandes.

Significado da distância entre RMSE e MAE: Quanto maior for a diferença entre o RMSE e o MAE, maior é a presença de erros pontuais grotescos/outliers de previsão. Se o RMSE for muito próximo do MAE, significa que a magnitude dos erros é homogênea e bem distribuída.

- Evidência Prática no Projeto:

No Random Forest Regressor, obtivemos MAE = 4,15 pontos e RMSE = 5,74 pontos.

Na Regressão Linear Múltipla, obtivemos MAE = 4,62 pontos e RMSE = 5,99 pontos.

A pequena diferença entre o RMSE e o MAE (uma variação de apenas ~1,3 a 1,5 ponto) confirma que os modelos não estão cometendo erros discrepantes graves em nenhum paciente específico; as falhas de previsão são pequenas, contidas e distribuídas uniformemente ao redor da média.

Pergunta 2: Qual é o papel da Validação Cruzada (5-Fold Cross-Validation) no processo de avaliação? Por que não podemos confiar apenas em uma única divisão de Treino e Teste (Hold-out)?

- Fundamentação Teórica:

Risco do Hold-out Isolado (Treino/Teste simples): A divisão simples de 80/20 pode ser vítima do "azar" ou da "sorte" da amostragem aleatória (gerando um conjunto de teste que seja atipicamente fácil ou atipicamente difícil de prever).

Mecânica da Validação Cruzada (5-Fold CV): O conjunto de treinamento é fatiado em 5 blocos (folds) iguais. O modelo é treinado e validado 5 vezes consecutivas: a cada rodada, 4 blocos são usados para treinar e 1 bloco para validar, alternando a fatia de validação.

Objetivo: O score final do R^2_{CV} é a média das 5 rodadas. Isso reduz a variância da estimativa de performance e garante que todo o dataset de treino foi testado, oferecendo um indicador de generalização muito mais estável e isento de viés de amostragem.

- Evidência Prática no Projeto:

Na Regressão Linear Múltipla, o R^2_{CV} na validação cruzada foi de 83,36%, enquanto o R^2 no teste final foi de 81,62%.

No SVR, o R^2_{CV} foi de 83,10% e no teste foi de 82,15%.

A proximidade extrema entre as médias da Validação Cruzada de 5 dobras e o teste independente prova estatisticamente a estabilidade do pré-processamento e a ausência de viés na amostragem.

Pergunta 3: Se vocês tivessem que apresentar os resultados para a diretoria médica de um hospital (stakeholders não técnicos), como explicariam o MAPE (%) e por que ele é mais intuitivo que o R^2?

- Fundamentação Teórica:

O R^2 (Coeficiente de Determinação) é uma métrica estatística relativa que mede a proporção da variância total da variável resposta explicada pelas features (ex: "83% da variação do score de saúde é explicada pelos hábitos"). Para um leigo ou diretor, essa definição abstrata de "variância explicada" é difícil de interpretar na prática clínica.

O MAPE (Mean Absolute Percentage Error) mede o erro absoluto como um percentual do valor real:

$$\text{MAPE} = \frac{1}{n} \sum \left| \frac{y - \hat{y}}{y} \right| \times 100\%$$

Por que o MAPE é intuitivo para o negócio: Ele responde diretamente à pergunta do gestor em porcentagem real: "Em média, em quantos por cento o modelo erra a nota de saúde do paciente?

- Evidência Prática no Projeto:

O Random Forest atingiu MAPE = 5,52%, o SVR = 5,54% e a Regressão Linear = 5,74%.

Para a diretoria médica, apresentamos o MAPE porque ele traduz o erro para a escala percentual do dia a dia. Dizer que o nosso modelo possui um MAPE de 5,5% significa que, ao avaliar um paciente cuja nota real de saúde seria 80 pontos, o sistema prevê um valor com uma margem de erro de apenas ~4,4 pontos para mais ou para menos. É uma taxa de precisão de 94,5%, ideal para um sistema de triagem preventiva de baixo risco.

10.4 Toma de Decisão e Recomendação

Pergunta 1: Considerando que a Regressão Linear é muito mais simples e rápida de computar, o ganho de performance obtido com o Random Forest ou SVR justifica a perda de interpretabilidade do modelo?

- Fundamentação Teórica:

Princípio da Parsimônia (Navalha de Ockham): Diante de dois modelos com capacidade preditiva similar, a solução mais simples deve ser preferida.

Trade-off Ganho Marginal vs. Complexidade: Na área da saúde e em aplicações corporativas auditáveis, substituir um modelo transparente ("caixa-branca") por um modelo opaco ("caixa-preta") só se justifica se houver um salto expressivo de acurácia. A Regressão Linear Múltipla permite que médicos e gestores interpretem diretamente os coeficientes (β_i), sabendo exatamente o impacto isolado de cada hábito de vida na nota do paciente.

- Evidência Prática no Projeto:

O Random Forest Regressor atingiu $R^2 = 83,08\%$ ($MAE = 4,15$) no teste, enquanto a Regressão Linear Múltipla alcançou $R^2 = 81,62\%$ ($MAE = 4,62$).

O ganho de performance do Random Forest foi de apenas 1,46 ponto percentual no R^2 e uma redução de erro de apenas 0,47 ponto no MAE. Esse ganho marginal mínimo não justifica abrir mão da interpretabilidade médica e da transparência dos coeficientes da Regressão Linear.

Pergunta 2: Se este modelo fosse colocado em produção para prever dados em tempo real (milhares de requisições por segundo), qual algoritmo vocês recomendariam considerando o custo computacional de predição?

- Fundamentação Teórica:

Complexidade de Inferência em Produção (Latência e Custo de Nuvem):

Regressão Linear Múltipla: A previsão em tempo real é uma operação algébrica direta de produto vetorial ($O(p)$), executada em fração de microssegundos com consumo mínimo de CPU e memória RAM.

Random Forest Regressor: Para cada nova requisição, a informação precisa percorrer as ramificações de 100 árvores de decisão individuais ($O(M \cdot \text{profundidade})$) e calcular a média das respostas, gerando alta latência sob tráfego intenso.

Support Vector Regressor (SVR): Exige calcular a função de Kernel RBF em relação a todos os vetores de suporte armazenados na memória.

- Evidência Prática no Projeto:

No nosso código, o tempo de treinamento/inferência da Regressão Linear Múltipla foi de apenas 0,0022s, enquanto o Random Forest levou 0,4185s (cerca de 200 vezes mais lento).

Para servir APIs de alta volumetria em tempo real (milhares de requisições/s em aplicativos mobile ou prontuários eletrônicos), a Regressão Linear Múltipla é a única opção viável de infraestrutura.

Avaliando os três algoritmos sob a ótica da Ciência de Dados aplicada:

O Random Forest alcançou a maior precisão absoluta ($R^2 = 83,08\%$ e $MAE = 4,15$), sendo a nossa recomendação para cenários de suporte à decisão diagnóstica onde a precisão pontual é a única prioridade.

No entanto, para a implantação prática em larga escala, a nossa recomendação principal é a Regressão Linear Múltipla. Pela Navalha de Ockham, ela entrega $81,62\%$ de R^2 (desempenho quase idêntico ao Random Forest), porém é 200 vezes mais rápida, possui custo de servidor quase nulo e oferece explicabilidade médica total, permitindo que médicos e pacientes entendam exatamente o peso de cada hábito na sua nota de saúde.

Recursos

Referências Relevantes

- **Repositório do Código**: https://github.com/thibastos0/ML_Health_LifeStyle

Fechar